

A game of refactoring. Studying the impact of gamification in software refactoring

Leonard Elezi, Sara Sali, Serge Demeyer, Alessandro Murgia, Javier Pèrez
University of Antwerp, Belgium
{leonard.elezi,sara.sali}@student.uantwerpen.be,
{serge.demeyer, alessandro.murgia,}@uantwerpen.be, javipeg@gmail.com

ABSTRACT

Software refactoring is an essential skill for developers. It improves the quality of the software and reduces maintenance costs in the long run. In this paper, we investigate the impact that gamification can have on the refactoring process and on the usability of the existing refactoring tools. For this reason we created CodeArena, a gamification system that tracks and rewards refactorings during development. We used CodeArena in an exploratory study which involved 12 students of computer science. Then, we collected the feedback provided by these students via a survey. Although the gamification had less effect than expected, our findings can be useful to practitioners interested in promoting refactoring tools via gamification.

CCS Concepts

• **Software and its engineering** → **Integrated and visual development environments**; *Software maintenance tools*; Software evolution;

Keywords

Gamification, Refactoring, Eclipse, CodeArena, Survey

1. INTRODUCTION

Refactoring is performed in order to improve software quality and developer productivity [12]. Furthermore, if no refactoring is performed, the cost of software maintenance may increase due to incurred technical debts [8]. Developers are quite aware of the refactoring benefits [15]. Yet, they hang back in the adoption of tools able to support refactoring (e.g. Eclipse)[14].

We believe that the lack of motivation and experience may hinder developers in the adoption of refactoring and refactoring tools. To address this problem, we propose the adoption of gamification in a refactoring environment, namely the application of game elements to a non-game environment [13].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

XP '16 Workshops, May 24 2016, Edinburgh, Scotland Uk

© 2016 Copyright held by the owner/author(s). Publication rights licensed to ACM. ISBN 978-1-4503-4134-9/16/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2962695.2962718>

To apply gamification in developer environments (IDEs), we propose CodeArena, a gamification tool that rewards developers when they refactor the code using a refactoring tool. For each refactoring, the tool assigns a score. The more a developer refactors, the more they increase their score. All developer scores are collected together and displayed in a leaderboard. As a result developers can compare their results with their peers, and may be encouraged to refactor.

In this paper, we perform an exploratory study to verify if gamification can be applied to refactoring. We present our experience with designing and implementing CodeArena. Moreover, we also report on the feedback we received from the users. Our study is meant to be a complement on current studies on refactoring tools adoption and not a replacement. The research questions that lead our study are:

- **RQ1:** Can we incorporate game elements such as score and leaderboards in a non-intrusive software refactoring tool?
- **RQ2:** How do game elements affect the refactoring process?
- **RQ3:** Are developers willing to use a gamified software refactoring tool?

The contributions of this paper are twofold:

1. We develop a gamified tool that helps developers refactor.
2. We report the lessons learned on how a gamified refactoring tool is received by developers.

The rest of this paper is organized as follows: Section 2 contains background on the design of CodeArena and the rationales behind the game elements used. Section 3 covers how the experiment was performed. Section 4 gives a description of who were the intended users. Section 5 contains the results of the experiment and the survey. Section 6 discusses threats to validity. We end with conclusions in Section 7 and acknowledgments.

2. CODEARENA

CodeArena is a gamified tool that rewards developers when they use Eclipse's refactoring commands as shown in Table 1. Figure 1 shows the overview of the the main components of CodeArena: an Eclipse plugin and the web application. The Eclipse plugin keeps track of Eclipse's refactoring commands that have been issued by the developer (see section 2.2). The web application calculates the score points, shows the refactorings of each user in a feed and compiles a leaderboard (see section 2.1).

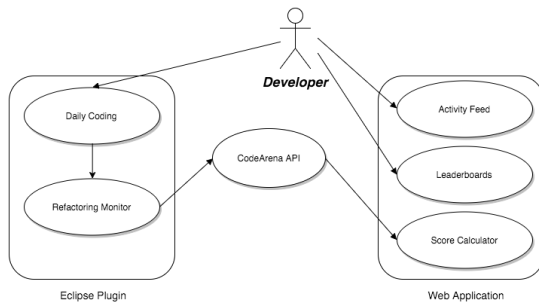


Figure 1: CodeArena Overview

2.1 Web Application

The web application consists of three components: the activities feed, the score calculator, and the leaderboard. The activities feed shows all the refactoring activities of the other users. For each user it displays the storyline of the refactorings performed (Figure 3), and points, level and progress to this level (Figure 2). The user's progress is the percentage completed on the current level (green section in Figure 2 marked with 20%). The score calculator receives the refactoring performed by the user, finds the corresponding type as shown in Table 1, and updates the user's points. The leaderboard ranks the users according to their points. For our purposes, it gets updated every time the user's score changes.

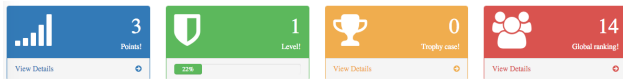


Figure 2: User's Progress

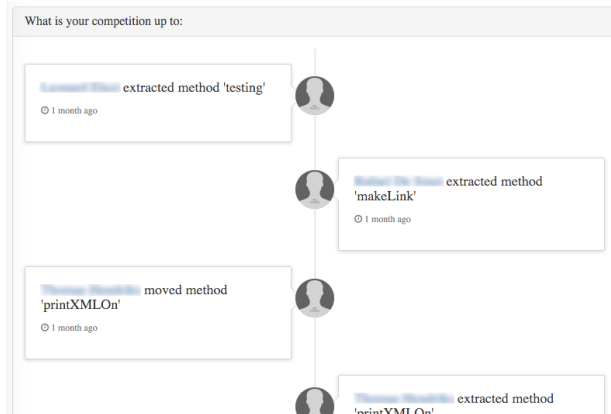


Figure 3: Activities Feed

2.2 Eclipse plugin

CodeArena is an Eclipse plugin composed of two views: a leaderboard view equivalent to the web application leaderboard, and a profile view showing user credentials and their progress with a progress bar as shown in Figure 4. Both views, while visible, are refreshed every time the user conducts a refactoring. The plugin plays a victory sound every-time the user is first on the leaderboard, and an info dialog is

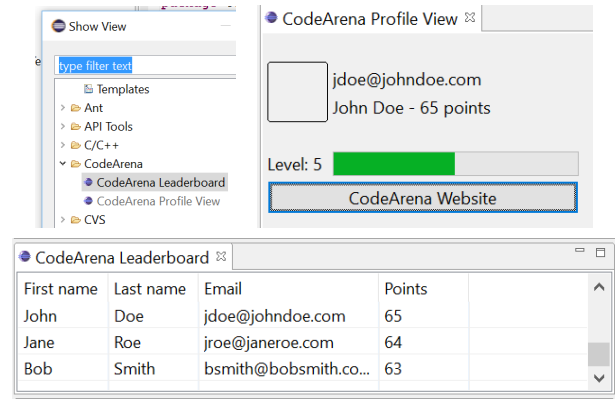


Figure 4: CodeArena Eclipse Views

shown shortly when a new level is reached (Figure 5). This plugin was created using the Eclipse API as documented in the official website [10].

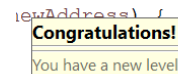


Figure 5: New Level Unlocked Notification

The plugin listens to the refactoring events, and sends all the information to the web application through the API. This information includes date and time, logged in user credentials, project name, and description of the refactoring. An example of description is: "Rename local variable 'blue'", which shows that the variable named blue was renamed. Table 1 lists the refactoring commands in Eclipse Luna, the version we used for building CodeArena.

2.3 Gamification Elements

Various gamification techniques have been applied to software engineering [16]. In CodeArena we implemented the following:

- **ActivitiesFeed:** Sometimes called a news feed. It serves to raise the awareness of participants as shown in [7].
- **Points and Leaderboards:** It has been shown that giving points and levels motivates users to contribute more content to an internal company social-network [17]. The leaderboard is compiled by ranking the developer's total points. In this way, CodeArena serves as a ground for developers to compete with each other.
- **Progress Bars:** It provides information about the current status of a player towards a goal.

All these elements provide feedback and clear goals, both these crucial mechanisms in a gamification system.

Points scored by a developer are retrieved via Eclipse. Eclipse defines the types of refactoring in the refactoring menu, the commands of which are shown in Table 1. We give each type of refactoring an arbitrary number of points shown again in Table 1. The number of points is defined (by the first two authors) according to the level of the difficulty of the refactoring. When a developer performs a refactoring through the Eclipse refactoring menu command, they get the points of that type of refactoring. The leaderboard is compiled by ranking the developer's total points.

In this way, CodeArena serves as a ground for developers to compete with each other.

Table 1: Refactorings and Points

Refactoring Type	Points
Extract interface	5
Move	5
Extract class	3
Extract method	3
Extract super class	3
Convert anonymous class to nested	2
Inline	2
Move type to new file	2
Change method signature	1
Convert local variable to field	1
Delete resource	1
Encapsulate field	1
Extract constant	1
Extract local variable	1
Generalize declared type	1
Infer generic type argument	1
Introduce faction	1
Introduce indirection	1
Introduce parameter	1
Introduce parameter object	1
Push down	1
Push up	1
Rename	1
User supertype where possible	1

3. EXPERIMENTAL SETUP

At the end of the fall semester of each academic year the computer science undergraduates of the University of Antwerp have to follow a software refactoring lab session, part of the software engineering course [4]. The purpose of this lab is to show how, by performing a series of small, manageable refactorings, one can adjust the design of a software system such that the implementation of new features can be resumed.

The software system to be refactored in the lab is the Local Area Network Simulation [9]. The project is C++ based and Java based. The Java based project used for the experiment can be downloaded from [3]. The type of refactorings and the steps to be followed in order to perform them are outlined in the document SRe2LICRefacLabo inside [3]. There are 4 main tasks that should be performed: (i) eliminating duplicate code with “Extract Method”, (ii) moving behavior close to the data performed by moving methods from one class to another, (iii) eliminating code navigation, (iv) transforming self type checks. Even though this is an individual exercise, discussions and collaboration are encouraged.

We used the lab session scheduled for December 2015 to perform our experiment. One group of twelve participants were present for the session. The experiment lasted for 2 hours. The twelve students started and finished at the same time. The experiment was conducted with the following steps:

1. We setup the environment with Eclipse and the CodeArena plugin.

2. We introduce the twelve participants to CodeArena and how they can create their credentials.
3. The participants use the credentials to activate the plugin.
4. We show the leaderboard in a big screen for the whole participants to see.
5. The participants import the LAN Simulator Java project in Eclipse and then they follow the refactoring steps outlined in the documents, with the occasional clarifying questions.
6. In the last 15 minutes we distribute the link to the survey for the participants to fill.

Google Form is used to compile the questions of the survey. Questions were organized in four groups: refactoring, gamification, environment (Eclipse, Java) and CodeArena. A link to the survey questions and results is given in [11].

4. TARGET USERS

We assessed the background of the participants with some questions in the survey. We asked about refactoring and gamification knowledge, Java and Eclipse experience, and the level of competitiveness in gamification platforms.

There were 12 participants in the experiment, all of them males. They were mostly novice developers. They were taking a refactoring class for the first time in their education. The average knowledge about Java, Eclipse, and refactoring is summarized in Table 2. The rating was on a scale from 1 (*beginner*) to 6 (*expert*). Their average rating is below 3.

Table 2: Student Knowledge

Topic	Refactoring	Java	Eclipse
Average	2.42	2.75	2.92

Most students are well acquainted with the concept of gamification, especially through StackOverflow. Popular platforms with game elements are Duolingo [1], Foursquare [2], Yelp [6], and StackOverflow [5]. Most of the students, 8 out of 12, say they have used at least one of these before. StackOverflow is on every positive answer. Only one of the students plays motivational games in a work/study environment with other colleagues.

To test more on motivational games experience, we ask “On completing a task successfully, do you reward yourself?”. This was an open and personal question and they answered in different manners. All answers referred to different types of rewarding systems. The majority prefer taking a break and performing leisure activities such as: listening to music, watching a movie, browsing social media and playing a game. One student replied that he does not reward himself, while another one forces himself to work more.

We compared the user’s ratings before and after the experiment for a better evaluation of CodeArena. On a scale 1 to 6 (six is the maximum), the students rated themselves an average of 2.9 on competitiveness and 4 on happiness before the experiment. This information is summarized in Figure 7.

5. RESULTS

In this section, we provide the results of our experiment and survey, and we answer our research questions.

Table 3: Reasons to complete assignment

Answers	Chosen times
I wanted to see how the system worked	7
I just wanted to be done with it	3
What else was I supposed to do?	1
I wanted to win at all costs.	1

5.1 Game elements and interference

RQ1: Can we incorporate game elements such as score and leaderboards in a non-intrusive software refactoring tool?

Part of this question was positively answered by developing CodeArena. We incorporated game elements in this tool as described in Section 2.

On one hand, the tool is non-intrusive. The purpose is not to invite users to refactor infinitely but to finish their job and get rewarded for the good practices. In our survey, we ask the students to rate on intrusiveness of the tool, and if they ever thought to disable CodeArena. The tool scored 4.6 on the scale 1 (*Interfering*) to 6 (*Ghost*) and none of the students felt interference to the point of disabling it.

On the other hand, having a non-intrusive tool had as a cost the limited number of game elements in the tool. Among students, 4 out of 12 were not aware of the notification received for having completed a new level(Figure 5). 4 students may not be a big number, yet, all of them had level 5 at the end of the experiment, and the notification has showed up four times for each. The chosen short time-span of the notification to keep the tool non-intrusive resulted in one game element less for these 4 students.

To conclude, **we can incorporate game elements in a non-intrusive software refactoring tool.**

5.2 Effects

RQ2: How do game elements affect the refactoring process?

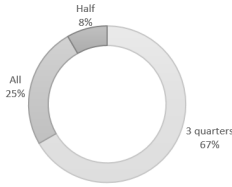


Figure 6: Assignment Completion

Students finished the biggest part of the assignment (Figure 6). However, this result being typical and expected, we can not determine if the tool added any impact in this achievement. Given four choices to answer: *"Why did you want to finish the assignment?"* most students admit they were curious about CodeArena and automatic refactoring, as seen in Table 3.

The students were also asked how much their rewarding systems motivated them, and how much CodeArena motivated them. The respective average values are 3.58 and 3.67 on a scale of 1 to 6. There is not a big difference but this might explain the low values given to CodeArena in general.

We can thus infer that **the new tool sparked curiosity, and somewhat motivated the students to finish their**

Table 4: CodeArena Experience

Answers	Chosen times
My friends were cheating	8
I was constantly checking the leaderboard	7
I was discouraged by the leaderboard	3
I checked occasionally	1

lab assignment.

On a multiple choices question, the users rated the experience as shown in Table 4, and one user gave an open answer: *"Since you get score every time you do any refactoring, without further checks, the leaderboard is essentially meaningless."* To clarify this last comment, we describe the loopholes (cheats) found by the students during the experiment.

1. **Undo/Redo refactoring.** When a refactoring is rolled back, it generates a change event in Eclipse. The Refactoring Listener provided by Eclipse API [10] consider this change as a normal refactoring command, and then CodeArena increases the score appropriately. However, the students considered this refactoring unfair.
2. **Extract/Inline refactoring.** This is similar to the refactoring Undo/Redo except for the fact that both activities have different descriptions and are two distinguishable refactoring commands.
3. **Move classes between packages.** One of the users accidentally moved a class from one package to the other. This generates a refactoring event with the description *"Move compilation unit"*. As seen in Table 1, the move operation is given 5 points, and this is what the user was awarded in this case.

All of these cases were seen by students as cheating and were exploited for higher rankings. We tried to discourage this behavior by giving less points to the quick refactorings such as extract/inline variable or renames. The users with most instances of Cheat 1, were ranked second and third at the end of the experiment, which discouraged the other participants. Yet, 8 out of 12 students still do not think that the system was fair, and 7 out of the same 12 think they deserved the points received. The fairness might indicate how the students feel about the points allocated to other students who they believed were cheating. Only 4 users admitted to have cheated (using the above mentioned methods), and 3 of them think that they did not deserve the points received.

As result, we conclude that loopholes in the system may be exploited to win the game. Thus, **gamification elements may introduce drawbacks in the refactoring process.**

The rest of the results are summarized in Table 5.

5.3 Future use

RQ3: Are developers willing to use a gamified software refactoring tool?

We asked the students about their future intentions in using the tool and the summarized results are reported in table 6. One of the users mentions that CodeArena was motivating him to accomplish his task at the beginning, but this feeling faded away with time. Furthermore, in the comments and recommendation sections, the users mostly explain why they felt that the rewarding system was overly

Table 5: Summary Table

Topic	Answer
Motivation from progress bar in Eclipse	average 2.9 on a scale 1 <i>demotivated</i> to 6 <i>motivated</i>
Motivation from leaderboard in Eclipse	2 students say they didn't use it and the rest say they liked it or were motivated by it
Motivation from progress bar in the website	average 3.1 on a scale 1 <i>demotivated</i> to 6 <i>motivated</i>
Sounds	average 2.3 on a scale 1 <i>demotivated</i> to 6 <i>motivated</i>
Animation	average 2.8 on a scale 1 <i>demotivated</i> to 6 <i>motivated</i>
Profiles and friends rating	55% admit it has some impact on them
Friend's activity	7 out of 12 students say they liked it
Motivation from tool in general	average 3.67 on a scale 1 <i>demotivated</i> to 6 <i>motivated</i>
Competitiveness from tool in general	average 3.42 on a scale 1 <i>non competitive</i> to 6 <i>competitive</i>
Rating how helpful was the tool in general	7 out of 12 answered no and an average 2.8 on a scale 1 <i>not helpful</i> to 6 <i>helpful</i>

Table 6: Future Use

Reuse the tool in the future	average 3 on a scale 1 <i>unlikely</i> to 6 <i>very likely</i>
Post results on other social media	mostly no
Compete with friends	average 2.4 on a scale 1 <i>unlikely</i> to 6 <i>very likely</i>

generous or easy to cheat and how they would have liked it to be. Summarizing all comments, we conclude that **students are not willing to reuse CodeArena in their work environment**, share results on social media and compete with friends.

6. THREATS TO VALIDITY

Threats to construct validity happen when the measurements may not show what the experiment wanted to evaluate. In our study many thresholds (e.g. score, levels, level ranges) have been decided arbitrarily. However, other values could have also been suitable. The scoring system was arbitrarily designed. The points per refactoring as shown in Table 1 were assigned in a subjective manner. A deeper evaluation, for instance a survey by experienced developers, is needed. The progress across levels is debatable because it was adapted to the LAN Simulation project. This may have affected the behavior (and feedback) provided by students. Furthermore, the scoring system is too sensible

to simple activities that may not be considered refactoring, such as Undo and Redo. Thus a developer that performs many Undos/Redos may accumulate more points than another one that performs the same number of refactorings correctly. Finally, we have to consider also that CodeArena depends on Eclipse, and supports mainly Java. As a result, the feedback of CodeArena may be influenced by the knowledge and opinions that students have towards Eclipse and Java.

Threats to internal validity refer to confounding factors that may influence the obtained results. We made participants aware of the purpose of the experiment. As a result, students may have focused more on the gaming parts instead of the refactoring activity.

Threats to external validity refers to the generalization of the results. Our experiment involved only 12 students of the same university. Participants of the study were students with limited or no knowledge of Eclipse and the refactoring process. To increase the generalization of the results, the experiment should be performed using a higher amount of participants, preferably with developing experience.

7. CONCLUSIONS

There is a lack of adoption of tools able to support refactoring. To address this problem, we suggest the introduction of game elements in software refactoring tools to encourage developers in their adoption. For this reason, we developed CodeArena, a gamified tool for promoting the adoption of the refactoring capabilities of Eclipse. CodeArena pushes developers to refactor by creating a competitive environment where each refactoring is rewarded. To evaluate the user experience of CodeArena, we conducted an experiment on 12 undergraduate students.

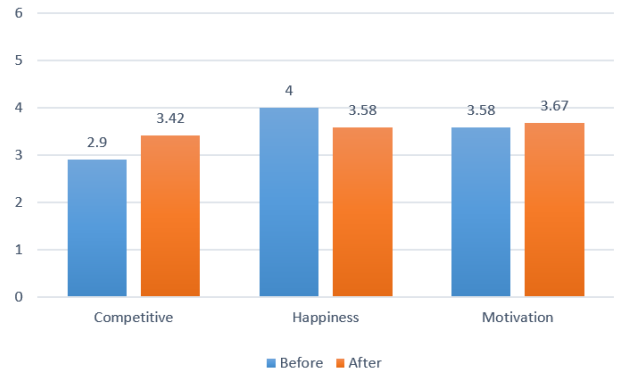


Figure 7: Compared Results

The results confirmed that it is possible to build a gamified non-intrusive software refactoring tool. However, the developers generally feel indifferent with regard to adopting the tool. On one hand, the tool was perceived as non-intrusive. On the other hand, the tool was not considered to have a real impact during the refactoring. This may depend on the fact that students considered the way CodeArena uses to assign refactoring points too generous, which may have lead some students to cheat in order to achieve high scores without focusing on the real purpose of the refactoring. Finally, we observed that students were initially highly motivated when using CodeArena. They learned the usage

of the tool, and they discovered the refactoring commands that Eclipse provides. However, we noticed a decline in their interests as soon as they completely explored how the system worked. The majority of the students were not willing to adopt CodeArena in the future, which may mean that the adoption of gamification in refactoring can backfire when the gamification layer is not suitably designed.

To promote the adoption of refactoring tools via gamification, we identified several limitations and drawbacks that need to be addressed:

- CodeArena is too greedy. It monitors and makes public every single refactoring in the refactoring activity view. In other locations and/or companies this may go against company culture and guidelines.
- Gamifying refactoring may cause participants to refactor just to get points, and not because it needs to be done. This is also an ongoing issue with gamification systems outside of refactoring.
- Showing the leaderboard on a big screen during the experiment has a downside. Some individuals get discouraged when they see that they are "losing" while other individuals just do the tasks for the points.

8. FUTURE WORK

CodeArena is a good starting point for studying the impact of gamification in software refactoring. However, it is rather simplistic. It is a gamification layer on top of Eclipse's existing refactoring API. As such it does not perform any quality evaluation on the types refactorings completed, and it does not evaluate if the software refactored still works. Furthermore, it trusts that the developers will run the tests after each refactoring to make sure they did not break the software. To tackle these drawbacks we suggest three solutions for future work:

- building the gamification layer in an already robust refactoring tool such as Ref-Finder. This makes it easier to find the different types of refactorings and prevents the Undo/Redo cheating.
- transforming the system in a reputation management system similar to StackOverflow's, where the refactorings are evaluated by other developers and not by the system. Thus the evaluation of a refactoring performed is left to the community.
- evaluating the refactoring based on successful run of tests. Give negative or no points in case a refactoring causes the tests to fail.

9. ACKNOWLEDGMENT

We would like to show our gratitude to the students following the 2015-2016 Software Engineering course for participating in our experiment and survey, and the University of Antwerp for allowing to conduct this research in the laboratory class.

References

- [1] Duolingo. <https://www.duolingo.com/>.
- [2] Foursquare. <https://foursquare.com/>.
- [3] LAN Simulation Java Project. <http://lore.ua.ac.be/Research/Artefacts/refactoringLabSession/RefactoringLabSessionsStudents.zip>. Accessed: 04-02-2015.
- [4] SE3BAC - Software Engineering. <https://www.uantwerpen.be/popup/opleidingsonderdeel.aspx?catalognr=1001WETSEN&taal=en&aj=2014>. Accessed: 04-02-2015.
- [5] Stack overflow. <http://stackoverflow.com/>.
- [6] Yelp. <http://www.yelp.com/>.
- [7] A. Begel and T. Zimmermann. Keeping up with your friends: function foo, library bar. dll, and work item 24. In *Proceedings of the 1st Workshop on Web 2.0 for Software Engineering*, pages 20–23. ACM, 2010.
- [8] L. A. Belady and M. M. Lehman. A model of large program development. *IBM Systems journal*, 15(3):225–252, 1976.
- [9] S. Demeyer, F. V. Rysseberghe, T. Girba, J. Ratzinger, R. Marinescu, T. Mens, B. D. Bois, D. Janssens, S. Ducasse, M. Lanza, et al. The lan-simulation: a refactoring teaching example. In *Principles of Software Evolution, Eighth International Workshop on*, pages 123–131. IEEE, 2005.
- [10] Eclipse.org. Eclipse documentation, February 2016. <http://help.eclipse.org/luna/index.jsp>.
- [11] L. Elezi and S. Sali. Codearena survey and answers, 2015. <https://goo.gl/e9tUwD>.
- [12] M. Fowler. *Refactoring: improving the design of existing code*. Pearson Education India, 2009.
- [13] J. Hamari, J. Koivisto, and H. Sarsa. Does gamification work?—a literature review of empirical studies on gamification. In *System Sciences (HICSS), 2014 47th Hawaii International Conference on*, pages 3025–3034. IEEE, 2014.
- [14] M. Katić and K. Fertalj. Towards an appropriate software refactoring tool support. In *WSEAS international conference on applied computer science*, pages 140–145, 2009.
- [15] E. Murphy-Hill, C. Parnin, and A. P. Black. How we refactor, and how we know it. *Software Engineering, IEEE Transactions on*, 38(1):5–18, 2012.
- [16] O. Pedreira, F. García, N. Brisaboa, and M. Piattini. Gamification in software engineering—a systematic mapping. *Information and Software Technology*, 57:157–168, 2015.
- [17] J. Thom, D. Millen, and J. DiMicco. Removing gamification from an enterprise sns. In *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*, pages 1067–1070. ACM, 2012.